

ENTERPRISE MOBILE APPLICATION DEVELOPMENT BOOTCAMP

AdventureWorks Mobile

iOS Capstone Project Requirements

TEAM FORMAT

Four-person Scrum teams

Swift • SwiftUI • REST • MVVM • XCTest • Git

Your mission: turn business needs into a tested, demonstrable iOS product.

1. Project Brief

AdventureWorks needs an iPhone application that gives employees and managers timely access to operational information while away from their desks. Your team will analyze the AdventureWorks API, create and refine a product backlog, design the user experience, implement a working iOS application, test it, and demonstrate it to stakeholders.

The requirements below describe business needs rather than completed user stories. Your team is responsible for deciding how to split them, identifying assumptions, asking Product Owner questions, writing acceptance criteria, and prioritizing work into an achievable MVP.

Success Definition

- A user can sign in and complete meaningful business tasks using live data from the AdventureWorks API.
- The application is understandable, responsive, accessible, and resilient when data is loading, unavailable, or empty.
- The codebase shows intentional architecture, collaboration through Git, and automated testing of important behavior.
- The team can explain its product decisions, technical tradeoffs, unfinished work, and lessons learned.

2. Team Organization

Each team has four members. All members are developers and must contribute code. Assign the following focus roles, then rotate them at least once during the project:

Focus role	Primary responsibility
Scrum Facilitator	Coordinates planning, stand-ups, backlog refinement, review, and retrospective.
Technical Lead	Guides architecture, integration choices, pull-request quality, and technical risk.
Quality & Accessibility Lead	Coordinates test strategy, acceptance checks, accessibility review, and defect triage.
Product & UX Liaison	Clarifies business value, maintains priorities, coordinates flows, and prepares stakeholder feedback.

Working Agreements

- Use a shared, public Git repository on GitHub.
- Protect the main branch and integrate work through reviewed pull requests.
- Use short-lived branches and commit in small, meaningful increments.
- No team member may be the sole owner of an application area; pair or review across feature boundaries.
- Track requirements, user stories, acceptance criteria, tasks, defects, and sprint status any way you wish.
- Agree on a Definition of Done before development begins.

3. Application Requirements

Required items form the baseline MVP. Choice items let your team shape the product. Complete at least two Choice requirements.

1	Secure sign-in and session management REQUIRED Employees need to sign in with credentials recognized by the AdventureWorks service. After successful authentication, the application must preserve the authorized session appropriately, attach the access token to protected requests, and provide a clear sign-out action. Invalid credentials and expired sessions must be handled without exposing secrets or technical details.
2	Operational dashboard REQUIRED Managers need a concise overview of current operations. The experience should make weekly sales, strong and weak product performance, low-stock products, and employee shift information easy to scan. The team must decide which information deserves the most prominence and how users move from a summary to supporting detail.
3	Employee directory and profile REQUIRED Users need to browse employees and open a useful profile for a selected employee. The list should help users distinguish people with similar names and should expose relevant department, title, shift, hire-date, or employment-history information when available.
4	Search, sort, or filter REQUIRED Users should not have to scan a long list to find a record. Provide at least two meaningful discovery controls within one or more major areas, such as partial-name search, department filtering, low-stock filtering, date-based sorting, or price sorting. The interface must make the active criteria apparent and allow them to be cleared.
5	Product catalog and details REQUIRED Employees need to browse AdventureWorks products and inspect details for a selected product. Present business-relevant content such as product number, summary, price, color, warranty, model information, or image data when available. Missing optional data must not produce a broken layout.

6

Inventory awareness REQUIRED

Operations staff need to see product quantities by location and identify stock that is at or below its reorder point. Users must be able to understand which product, location, shelf/bin, current quantity, and threshold are involved without mentally correlating separate screens.

7

Order exploration REQUIRED

Users need to review orders and inspect line-level detail. The application must support at least one order audience—individual customers or stores—and clearly present order identity, customer/store context, dates, products, quantities, prices, and totals.

8

Refresh and current-state feedback REQUIRED

Users need a deliberate way to request information refresh in at least two data-driven areas. The interface must distinguish initial loading, user-initiated refresh, successfully loaded content, an empty result, and a failed request. Repeated taps or navigation must not create confusing duplicate work.

9

Failure recovery and partial availability REQUIRED

A failed request must not crash the app or leave an endless progress indicator. The user should receive a plain-language explanation and a reasonable recovery action. Failure in one feature area should not unnecessarily block unrelated areas that can still operate.

10

Accessible and adaptable interface REQUIRED

Core tasks must be usable with Dynamic Type and VoiceOver. Interactive elements need meaningful labels and adequate touch targets, information cannot depend on color alone, and key screens must remain usable across supported iPhone sizes and orientations selected by the team.

11

Authorized inventory adjustment CHOICE

Operations staff may need to correct a product quantity at a specific location. Provide a guarded editing flow that shows the existing value, validates the proposed quantity, requires review or confirmation, sends the supported update, and clearly communicates success or failure.

12

Employee information update CHOICE

Staff may need to correct an employee's personal or employment information. Choose one supported update workflow, load valid reference data where needed, prevent clearly invalid submissions, and avoid silently overwriting information the user did not intend to change.

13

Token renewal CHOICE

Users with a valid refresh token should not be forced to sign in again solely because the access token expires. The application should attempt one safe renewal and retry the interrupted request. If renewal fails, it must return the user to a secure signed-out state without creating an infinite retry loop.

14

Local continuity CHOICE

Users should retain limited value during a temporary network outage. The team may cache selected read-only data, preserve recent records, or provide another justified continuity feature in one or more major areas. The interface must distinguish stored information from newly retrieved information and avoid presenting stale data as current without context.

4. Technical Constraints

Area	Constraint
Platform	Native iOS application written in Swift; use SwiftUI for the primary user interface.
API	Use the AdventureWorks API service at https://api.bootcampcentral.com/ . Swagger documentation interface is available at https://api.bootcampcentral.com/swagger/index.html . Do not replace live integration with hard-coded production data.
Concurrency	Use Swift concurrency (async/await) for asynchronous network work and keep UI state updates safe.
Models	Create typed Codable models for the API responses used by the application.
Architecture	Separate views, presentation/state logic, and data access. Use MVVM with a reusable API client in services or repositories, or obtain instructor approval for a comparable design.
Security	Do not commit credentials or tokens. Store sensitive session material using an appropriate secure mechanism. Logs and alerts must not reveal secrets. No screens shall be accessible without logging in.
Testing	Include automated unit tests for important business/presentation behavior and network-related logic using mocks, stubs, or injected test doubles. Include at least one UI test of a critical path.
Source control	Use feature branches, pull requests, peer reviews, and a clean main branch that builds throughout the project.
Dependencies	Use Swift Package Manager for approved third-party packages. The team must be able to justify each dependency.
Build	The submitted project must build and run from a clean clone using the documented supported Xcode/iOS versions.

5. Scrum and Planning Deliverables

- A prioritized product backlog derived from the requirements in this document (export or screenshot).
- Sample user stories that identify a user, capability, and value; large items split to fit the team's sprint cadence.
- Acceptance criteria for stories selected into a sprint, including unhappy paths and edge cases.
- A Definition of Ready and a Definition of Done used consistently by the team.
- A final retrospective with specific improvements the team would carry forward.

6. Product Deliverables

- Working Xcode project and Git repository with meaningful history.
- README with setup, configuration, supported versions, credentials procedure, architecture summary, and known limitations.
- Test suite and a brief test report identifying coverage priorities and unresolved defects.
- Backlog/board export or screenshots showing the team's Scrum work.
- Brief technical presentation covering architecture, security, resilience, testing, and tradeoffs.
- Individual contribution summary from each team member.

7. Assessment

The capstone will be evaluated as a team product, with individual contribution and understanding also considered.

Category	Weight	Evidence of success
Product value and requirement coverage	25%	Coherent MVP; required workflows are complete; choices support a clear product direction.
API integration and resilience	20%	Correct authentication and decoding; thoughtful loading, empty, failure, retry, and partial-availability behavior.
Architecture and code quality	20%	Clear responsibilities, reusable data access, readable Swift, appropriate state management, and justified tradeoffs.
UX and accessibility	15%	Native, understandable flows; useful feedback; adaptable layouts; core accessibility support.
Testing and quality practices	10%	Valuable automated tests, acceptance checks, defect awareness, and a stable demo build.
Scrum collaboration and communication	10%	Refined backlog, visible teamwork, reviewed Git history, clear presentation, and honest retrospective.

8. Before You Begin

Complete these activities before committing to the first sprint backlog:

- Explore the Swagger specification and match business requirements to available operations and schemas.
- Identify assumptions and Product Owner questions; do not quietly invent business rules.
- Sketch the navigation model and the critical happy path before dividing screens among team members.
- Agree on architecture, naming, branching, reviews, coding standards, and Definition of Done.
- Identify the riskiest technical or product uncertainty and address it early.

Questions for the Product Owner

Your team is expected to ask questions. Useful topics include required dashboard priorities, supported order audience, data freshness, offline expectations, edit permissions, formatting rules, and the definition of an acceptable MVP.

Do not treat this handout as a screen checklist. Build the smallest coherent product that delivers value, then improve it through feedback.

Appendix - Logins

Below are valid logins for the API:

Username	Password
ken0	295847284
terri0	245797967
roberto0	509647174
rob0	112457891
gail0	695256908
josef0	998320692
dylan0	134969118
diane1	811994146